

6. Multimodal Video Analytics Voice Interaction

6. Multimodal Video Analytics Voice Interaction

1. Concept Introduction
 - 1.1 What is "Video Analytics"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
 2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring Offline Large Model
 - 3.1.1 Configuring LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring Model Interface (`large_model_interface.yaml`)
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is "Video Analytics"?

In the `largemodel` project, the **Multimodal Video Analytics** feature enables the robot to process a video and summarize its core content, describe key events, or answer specific questions about the video in natural language. This allows the robot to advance from only understanding static images to understanding the dynamic, temporal world.

The core tool of this function is `analyze_video`. When a user provides a video file and asks a question (such as "summarize what this video is about"), the system invokes this tool to process and analyze the video, and returns the AI's text response.

1.2 Brief Description of Implementation Principles

The core challenge of offline video analytics is how to efficiently process video data containing hundreds or thousands of frames. A mainstream implementation principle is as follows:

1. **Keyframe Extraction:** First, the system does not process every frame of the video, but extracts the most representative **keyframes** through algorithms (such as scene boundary detection or fixed-time interval sampling). This greatly reduces the amount of data that needs to be processed.
2. **Image Encoding:** Each extracted keyframe is sent to a vision encoder, just like in the visual understanding application, and converted into digital vectors containing image information.
3. **Temporal Information Fusion:** This is the biggest difference from single-image understanding. The model needs to understand the temporal order between these keyframes. Usually, a Recurrent Neural Network (RNN) or Transformer model is used to fuse the vectors of all keyframes, forming a "memory" vector that represents the dynamic content of the entire video.

4. **Q&A and Generation:** The user's text question is encoded and then cross-modally fused with this "memory" vector. Finally, the language model component generates a summary of the entire video or answers to specific questions based on this fused information.

Simply put, it **condenses a video into several key images and their order, then understands the entire story like looking at a comic strip, and answers related questions.**

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `analyze_video` function in this file defines the execution flow of the tool.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def analyze_video(self, args):
        """
        Analyze video file and provide content description.
        """
        self.node.get_logger().info(f"Executing analyze_video() tool with args:
{args}")
        try:
            video_path = args.get("video_path")
            # ... (Smart path fallback mechanism)

            if video_path and os.path.exists(video_path):
                # ... (Build Prompt)

                # Use a fully isolated, one-time context for video analysis to
                ensure a plain text description.
                simple_context = [{
                    "role": "system",
                    "content": "You are a video description assistant. ..."
                }]

                result = self.node.model_client.infer_with_video(video_path,
prompt, message=simple_context)

                # ... (Process result)
                return {
                    "description": description,
                    "video_path": video_path
                }
            # ... (Error handling)
```

2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)

The functions in this file are responsible for processing video files and passing them to the underlying model.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_video(self, video_path, text=None, message=None):
        """Unified video inference interface."""
        # ... (Prepare messages)
        try:
            # Based on self.llm_platform, determine which specific
            implementation to call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
video_path=video_path)
            # ... (Other online platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

    def _extract_video_frames(self, video_path, max_frames=5):
        """Extract keyframes from a video for analysis."""
        try:
            import cv2
            # ... (Video reading and frame interval calculation)
            while extracted_count < max_frames:
                # ... (Loop to read video frames)
                if frame_count % frame_interval == 0:
                    # ... (Save frame as temporary image)
                    frame_base64 = self.encode_file_to_base64(temp_path)
                    frame_images.append(frame_base64)
                # ...
            return frame_images
        # ... (Exception handling)
```

Code Analysis

The implementation of video analytics is one step more complex than image analytics, as it requires a critical preprocessing step in the model interface layer: frame extraction.

1. Tool Layer (`tools_manager.py`):

- The `analyze_video` function is the business entry point for video analytics. Its responsibility is clear: receive a video file path and construct a prompt for requesting a description of the video content.
- It initiates the analysis process by calling `self.node.model_client.infer_with_video`. Like the visual understanding tool, it doesn't care about the underlying model details at all, only responsible for passing the "video file" and "analysis instruction".

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_video` function acts as a dispatcher connecting the upper-level tools and the underlying model. It dispatches tasks to the corresponding implementation functions based on platform configuration (`self.llm_platform`).

- Unlike processing a single image, processing video requires an additional step. The `_extract_video_frames` method demonstrates the general logic for this step: it uses the `cv2` library to read the video and extract several (default 5) keyframes.
- Each extracted frame is treated as an independent image and is typically encoded as a Base64 string.
- Finally, a request containing multiple frame image data and analysis instructions is sent to the large model. The model performs comprehensive analysis on these consecutive images to generate a description of the entire video content.
- This "video-to-multiple-images" preprocessing is completely encapsulated in the model interface layer and is transparent to the tool layer.

In summary, the general flow of video analytics is: `ToolsManager` initiates analysis request -> `model_interface` intercepts the request and calls `_extract_video_frames` to decompose the video file into multiple keyframe images -> `model_interface` sends these images along with analysis instructions to the corresponding model platform based on configuration -> The model returns a comprehensive description of the video -> The result is finally returned to `ToolsManager`. This design ensures the stability and versatility of upper-level applications.

3. Practical Operations

3.1 Configuring Offline Large Model

3.1.1 Configuring LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in terminal:

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

2. Modify/confirm `llm_platform`:

```
model_service:                                #Model server node parameters
  ros__parameters:
    language: 'zh'                             #Large model interface language
    useolinetts: True                         #This setting is invalid in text
mode, can be ignored

# Large model configuration
llm_platform: 'ollama'                       # Critical: Ensure this is set to
'ollama'
```

3.1.2 Configuring Model Interface (`large_model_interface.yaml`)

This file defines which visual model is used when the platform is set to `ollama`.

1. Open the file in terminal

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interface.ya
ml
```

2. Find the ollama-related configuration

```
#.....  
## Offline Large Language Models  
# Ollama configuration  
ollama_host: "http://localhost:11434" # Ollama server address  
ollama_model: "llava" # Critical: Change this to your downloaded  
multimodal model, such as "llava"  
#.....
```

Note: Please ensure the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

3.2 Startup and Testing

1. Prepare video file:

Place a test video file at the following path:

```
/home/sunrise/yahboom_ws/src/largemodel/resources_file/analyze_video
```

Then name the video `test_video.mp4`

2. Start the `largemodel` main program:

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake up:** Say to the microphone: "Hello, yahboom."
- **Conversation:** After the speaker responds, you can say: `analyze the video content`
- **Observe logs:** In the terminal running the `launch` file, you should see:
 - 1. The ASR node recognizes your question and prints it out.
 - 2. The `model_service` node receives the text, invokes the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker.

Recommendation: It is recommended to use the minicpm-v:8b offline model. Different offline models have different strengths and capabilities, and users can pull models to try on their own.